Machine Learning (2026) 115:72

Author Unknown

***Abstract*—**

***Index Terms*—Federated learning · Gaussian process · Random feature · Bayesian method**

I. Introduction

Uncertainty estimation has been an indispensable part of many practical areas. For example, in simulations widely used for the development of aircraft, trains, and vehicles (Danquah et al., 2020) and the testing of safeguarding autonomous vehicles (Riedmaier et al., 2020), uncertainty estimation is needed to assess the uncertainty inherent in any simulation model (Riedmaier et al., 2021). Researchers have also been using Bayesian optimization to speed up material design with nanoparticles (Xia et al., 2023; Iyer et al., 2020, 2019), where uncertainty estimation is used to balance exploitation and exploration. In machine-assisted decision making (Begoli et al., 2019), it provides the trust needed to inform decisions.

Mahdi Karami and Guojun Zhang: The work was done at their previous affiliation, Huawei Noah's Ark Lab. Editor: Gang Niu. Extended author information available on the last page of the article 1 3 72 Page 2 of 36 Machine Learning (2026) 115:72 Gaussian process (GP) has been an especially popular model that provides uncertain estimates (Verrelst et al., 2013; Li et al., 2021). GPs with deep kernel learning (Wilson et al., 2016a, 2016b) provide a good balance between expressiveness and simplicity to represent model uncertainty. At one end of the spectrum, most traditional models such as traditional neural networks do not capture any uncertainty, but are simple and scalable. At the other end of the spectrum, Bayesian neural networks express a full distribution over all weights of neural networks, but exact inference tends to be infeasible. In between, a GP with a deep kernel consists of a deep neural feature extractor with a distribution over the last layer that facilitates exact inference. Since the last layer is the most important for prediction, representing its uncertainty is often sufficient to capture most of the uncertainty of a model (Gawlikowski et al., 2022; Kristiadi et al., 2020).

However, extending GPs to the federated learning (FL) setting itself is challenging. In FL, we seek to train a model in a distributed way across several clients without any data leav ing the clients to preserve privacy. Several works have explored distributed GPs (Deisenroth & Ng, 2015; Yin et al., 2020) and federated GPs (Achituve et al., 2021; Kontoudis & Stil well, 2023, 2024). While distributed GPs are designed to improve scalability, they pose an important privacy risk since sharing kernels either implies sharing data or sharing pairwise data similarity. In contrast, federated GPs including pFedGP (Achituve et al., 2021),DECapx-GP (Kontoudis & Stilwell, 2023), and DEC-gapx-GP (Kontoudis & Stilwell, 2024) share only the hyperparameters of kernels while learning local GP posteriors that are never shared. This personalized approach reduces privacy risks, but the local GPs do not benefit from other client information beyond the shared kernel hyperparameters. Thus, its accu racy suffers when generalization is desired or new clients come without training data. For example, a hospital that only collected chest X-rays of lung emphysema patients may also want their model to help diagnose rib fractures with the same type of input. However, per sonalization would prevent the model from generalizing to other ranges. Moreover, new clients with few training data cannot effectively train predictors with local GPs, even if they do not need generalization.

To address such challenges, we propose a new federated GP technique called Federated Bayesian Neural Regression (FedBNR) that learns a global federated GP. We first define a unifying random kernel (URK) which allows random feature approximation of both sta tionary and non-stationary kernels. FedBNR with URK avoids kernel sharing by working directly in the feature space and sharing scatter matrices, reducing privacy risks and decreas ing the computational complexity to linear in the amount of data. Since FedBNR maintains a global posterior, it makes full use of different clients' information, and performs better for generalization and new clients. The approach is demonstrated on real world regression data sets where we achieve statistically significant improvements over prior techniques both in terms of point and uncertainty estimation. We further extend our algorithm with differential privacy to provide formal privacy guarantees. The main contributions are: \textbullet New random feature approximations of non-stationary kernels enabled by URK, and their concentration guarantees. \textbullet A new federated GP technique FedBNR, which achieves better generalization perfor mance in both accuracy and uncertainty calibration due to its global posterior.

1 3 Page 3 of 36 72 Machine Learning (2026) 115:72 The paper is structured as follows. In Sect. 2, we review essential background knowledge of GP, FL, and related works. In Sect. 3, we introduce URK and its theoretical guarantees, pro pose FedBNR based on URK, and extend FedBNR with differential privacy (DP). Sect. 4 details synthetic and real-world experiments. Finally, Sect. 5 concludes the work.

II. Background

Notation. We use the following notation throughout the paper. X, X' ∈Rp×n, x, x' ∈Rp are input matrices or vectors. n is the number of data points and p is the number of features. y ∈Rn is the target vector. x∗, y∗ indicate vectors that have not been seen by the models. σ ∈R+ is the standard deviation

of the additive Gaussian noise in the models we men tion later. I is the identity matrix and 0 is the zero vector. Their dimensions can be inferred from the context. ϕ : Rp →Rd denotes a d dimensional basis function, and Φ = ϕ(X) denotes the corresponding features of X. k : Rp × Rp →R denotes a kernel function, and K = k(X, X) is the corresponding kernel Gram matrix. w ∈Rd are the weights of the linear layer in a Bayesian linear regression model. λ ∈R+ is the prior standard deviation on w. E[·], Cov(·, ·), and Tr(·) are the expectation, covariance, and trace function. Any symbol with a c subscript is a local version of the original symbol, held by client c.

2.1 Gaussian Process and Random Fourier Feature A GP can be viewed as an infinite dimensional Gaussian distribution over function objects f(·). If restricted to a finite set of points of interest X on the support, a GP becomes a multi-dimensional Gaussian distribution f(X), providing distributional information at these places. Formally, GP regression with additive Gaussian noise is established as y = f(x) + ϵ, where ϵ ∼N(0, σ2). After a prior over f(·) is specified, the likelihood, posterior, and prediction can be computed as follows: Prior: f(·) ∼GP(0, k(·, ·)) (1) Likelihood: (y|X, f(·)) ∼N(f(X), σ2I) (2) Posterior: (f(·)|X, y) ∼GP(¯m(·), k'(·, ·)), where (3) ¯m(·) = k(·, X)(K + σ2I)-1y,

(4) k'(·, ·) = k(·, ·) -k(·, X)(K + σ2I)-1k(X, ·)

(5) Prediction: (y∗|x∗, X, y) ∼N(¯m(x∗), σ2 + k'(x∗, x∗)) (6) The runtime complexity of GP is O(n3) due to K-1. Thus in practice, full GPs are often infeasible, and approximations are needed for scalability. The performance of GP models highly depends on the kernel function k(·, ·), and hyperparameters of this function can be learned by maximizing log Pr GP (y|X) = -y⊤(K + σ2I)-1y -log |K + σ2I|. 1 3 72 Page 4 of 36 Machine Learning (2026) 115:72 A valid kernel function is any positive definite function and can always be decomposed into the inner product of some basis functions k(x, x') = ϕ(x)⊤ϕ(x') (Cuturi, 2009). An important advantage of working in the function space (i.e., with kernels) is that kernel func tions can correspond to infinite features (i.e., the corresponding ϕ(x) must map x to infi nitely many linearly independent features) without paying a price in complexity. Popular kernels, such as the Gaussian kernel k(x, x') = exp(-||x -x'||2/2γ2) (γ ∈R+ is a kernel hyperparameter), tend to induce infinite dimensional features, providing significant model capacity.

If the kernel has finite features (i.e. ∃ϕ(x) that maps x into finitely many features), a GP degenerates into Bayesian linear regression (BLR) in the feature space with parametrized model f(·) = w⊤ϕ(·). Given a prior over w, the likelihood, posterior, and prediction are: Prior: w ∼N(0, λ2I) (7) Likelihood: (y|X, w) ∼N(w⊤Φ, σ2I) (8) Posterior: (w|X, y) ∼N(¯w, A-1), where (9) ¯w = σ-2A-1Φy and A = σ-2ΦΦ⊤+ λ-2I

(10) Prediction: (y∗|x∗, X, y) ∼N(ϕ(x∗)¯w, σ2 + ϕ(x∗)⊤A-1ϕ(x∗)) (11) Recall ϕ(X) ∈Rd. Then the complexity of BLR is O(nd2 + d3) due to A-1. Hyperparameters of ϕ can be learned similarly by maximizing log Pr BLR (y|X) = -n log σ2 -log |λ2I| -log |A| -||y -Φ ¯ w||2 -λ-2 ¯ w⊤¯ w. It is very important to note that Eqs. 6 and 11 are identical, with only the order of matrix multiplication changed, resulting in different complexity and model interpretation.

Random Fourier features (RFF) (Rahimi & Recht, 2007) can be used to approximate infinite stationary kernels with finite features based on Bochner's theorem: Theorem 2.1 (Bochner's theorem, e.g. (Rudin, 2017)). A continuous stationary kernel k(x, x') = ~k(x -x') = ~k(δ) on Rp is positive definite if and only if ~k(δ) is the Fourier transform of a non-negative measure.

Above, δ = x -x' and ~k is the kernel k redefined in terms of one argument that is the difference of the inputs x -x'. If the kernel is real-valued and properly scaled, its inverse Fourier transform p(ω) is also real-valued and is a proper probability distribution. Then a valid feature mapping is zω(x) = (cos(ω⊤x), sin(ω⊤x)), since ^ ~k(δ) = Rp p(ω) cos(ω⊤δ)dω = Eω[cos(ω⊤δ)] = Eω[zω(x)⊤zω(x')] The true expectation is then approximated by the empirical mean of zω(x)⊤zω(x') with multiple samples from p(ω), which enables recovering an infinite kernel with a finite set of features and working directly in the feature space.

1 3 Page 5 of 36 72 Machine Learning (2026) 115:72

2.2 Federated Learning v.s. Distributed Learning In federated learning (FL), there is always a collection of clients c ∈S that wish to col laborate, and each client holds their own data Dc = (Xc, yc) locally. We seek to train some machine learning model Mθ on these client data, with parameters θ. Conventionally, we would centralize all the data D = ∪ c∈S Dc, and learn the model globally. This simple approach becomes infeasible if the clients cannot directly share their data due to privacy concerns. FL is designed to specifically tackle this problem. In typical scenarios, a trusted central server is allowed to receive and send perturbed matrices that only contain limited information about raw client data such as model parameters.

Although FL seems similar to distributed learning (DL) where tasks are also distributed to client workers to exploit extra computational resources, FL focuses on settings that are very different from typical DL settings and thus faces unique challenges that DL algorithms cannot handle. First, DL algorithms are usually deployed on clusters, where communication between nodes are fast and reliable, thus algorithms with huge message throughput, like aggregating local gradients at each iterative step, are practical and feasible. However, in the cross-device setting of FL, client nodes are often millions of mobile devices that are not reliable and possess only limited computational resources. In this case, it is crucial to reduce the communication cost, so FL algorithms are often expected to aggregate local models or information that have evolved for more than one step or epoch of gradient descents from the previous global consensus, and a line of works (Konečnỳ et al., 2016; Sattler et al., 2019; Reisizadeh et al., 2020) are dedicated in improving the communication efficiency. This nature of clients further leads to data and system heterogeneity, since the server often, if not always, has no control over how data or computation resource is distributed. More importantly, FL algorithms focus on reducing privacy risks of clients, while it is not a con cern at all of DL algorithms. Hence, many DL

algorithms send risky messages with which other entities can easily retrieve the raw data of clients. A review with more details can be found in Appendix A.

III. Related Work

Distributed and Federated GPs Closely related to our work is the literature on distributed and federated GPs. Distributed GPs (Deisenroth & Ng, 2015; Yin et al., 2020) were initially proposed to improve scalability by partitioning the data into several machines since GPs that operate in the dual space scale cubically with the amount of data in the worst case. The product-of-experts framework has emerged as a popular technique to aggregate local GPs, including generalized product of experts (Cao & Fleet, 2014) and robust Bayesian committee machines (Deisenroth & Ng, 2015). Distributed optimization of hyperparam eters in GPs has also been explored using ADMM (Xie et al., 2019), statistical parameter matching (Yurochkin et al., 2019), or variational inference (Gal et al., 2014; Hoang et al., 2016). While those techniques do not ensure data privacy, recent work about federated GPs reduce privacy risks while training in a distributed way. This includes pFedGP (Achituve et al., 2021)and DEC-apx-GP (Kontoudis & Stilwell, 2023) which optimize the hyperpa rameters of a global kernel using either FedAvg or decentralized proximal ADMM. DECgapx-GP (Kontoudis & Stilwell, 2024) is an extension of DEC-apx-GP, which shares an additional dataset with all clients as extra training data. Note this violates the constraints of 1 3 72 Page 6 of 36 Machine Learning (2026) 115:72 FL, in the absence of a public dataset, since the additional dataset must consist of client's local data. These federated GP algorithms share only the hyperparameters of kernels, but learn local GP posteriors that are never shared. In another line of work, GPs have also been used to estimate correlations between clients in FL in order to actively select independent clients for aggregation (Tang et al., 2021). Our work differs from previous distributed and federated GPs by learning a global federated GP with better generalization performance.

Deep kernel learning and GP approximations There have been many works that commit ted to increasing the model capacity of GPs by incorporating deep neural networks (DNNs). Works of Hinton and Salakhutdinov (2007), Calandra et al. (2016) either pretrain a deep belief network or directly train it with a GP to extract first-step features before sending the data into the GP with conventional kernels. Wilson et al. (2016a, 2016b) build their algo rithm on top of Wilson and Nickisch (2015), Wilson et al. (2015), Titsias (2009), Hensman et al. (2013), Nickson et al. (2015), extending this idea with approximations for scalability and stochastic variational inference for classification tasks. Then Tran et al. (2019) studies the variance estimations of deep kernel learning models and proposes to use Monte-Carlo Dropout (Gal & Ghahramani, 2016) for better calibration. Ober et al. (2021) proposes to use Bayesian Neural Networks instead of deterministic DNNs to prevent over-fitting. Besides, Garnelo et al. (2018) designed a special architecture that makes it possible for DNNs to simulate GP behaviors. Damianou and Lawrence (2013) form GPs into a deep architecture that corresponds to a deep belief network based on GP mappings. To make GPs practi cal, one popular method is the inducing point approximation, where the joint GP prior of training points and inducing points are approximated (Quinonero-Candela & Rasmussen, 2005). Variants includes SoD (Quinonero-Candela & Rasmussen, 2005), SoR (Smola & Bartlett, 2000), DTC (Seeger et al., 2003), FITC (Snelson & Ghahramani, 2005), and PITC (Schwaighofer & Tresp, 2002). Later, KISS-GP (Wilson & Nickisch, 2015) gave another interpretation that inducing point approximations are equivalent to global GP interpolation, and it can exploit Kronecker structures (Saatçi, 2012). Another approximation method is random features (Rahimi & Recht, 2007; Sinha & Duchi, 2016; Oliva et al., 2016) that use randomized basis functions to approximate kernels. More information about scalable GPs can be found in this survey (Liu et al., 2020).

3 Federated Learning of Global GPs Personalized federated learning (PFL) algorithms can learn fine-tuned local models for each client and can automatically handle preference distribution skew (Zhu et al., 2021) (i.e., Pr(y|X) varies with clients). However, this comes at the cost of the generalization ability, especially when preference skew is not a concern and the clients want out-of-distribution generalization w.r.t. its training data. If a client is not capable of collecting enough data in the same range as their desired prediction, the accuracy of PFL local models could suf fer. For example, suppose the posterior of a GP with a stationary kernel is conditioned on local training data (x1, · · · , xnc) ⊂Bc, an infinite compact set, and (y1, · · · , ync). Due to non-i.i.d. clients, it is common that other clients c' have a different Pr(X) in the form that Bc ∩Bc' = Bc'. That is, training data of client c' is considered out-of-range for client c. Suppose client c wants to generalize to the range of all clients' training data ∪c∈SBc. The prediction of data points from other clients' range x' /∈Bc will gradually become the con 1 3 Page 7 of 36 72 Machine Learning (2026) 115:72 stant zero prior as x' moves further from Bc, regardless of what y' really is. Thus, this gener alization error is unbounded. However, a global GP does exhibit a uniformly bounded error (Lederer et al., 2019), since its posterior is conditioned on all clients' data. It means all the clients, whether or not possessing training data, can benefit from others' information in FL. With such motivation for learning global federated GPs, we derive our algorithm in the following sections. In Sect. 3.1, we propose a random feature formulation URK, which is more flexible and compatible to deep learning as it can recover non-stationary kernels. It also provides more model capacity and thus better accuracy compared to plain deep learning without random features. Then, we detail our algorithm FedBNR in Sect. 3.2, showing how a federated global posterior can be learned with URK. Finally, we extend FedBNR with DP in Sect. 3.3, providing formal privacy guarantees (Fig. 1).

3.1 GP Under Random Feature Maps

URK Definition Recall that a kernel function can be rewritten as the inner product of some basis functions k(x, x') =

ϕ(x)⊤ϕ(x’). We propose the following kernel and random fea ture approximation.

Let ω ∈Rd’ be any random variable or vector, and g : Rd’ × Rp -→Rd be any function that extracts d features out of each input with random weights ω. We construct the random basis functions z as zω(x) = g(ω, x). We define the true underlying kernel and its approxi mation, the unifying random kernel (URK) as: k(x, x’) = Eω[zω(x)⊤zω(x’)]

(12) ( )⊤( ) sm ω (x’) √m -1 sm ω (x) √m -1 ≈URKm ω,g(x, x’) := = ϕ(x)⊤ϕ(x’)

(13)

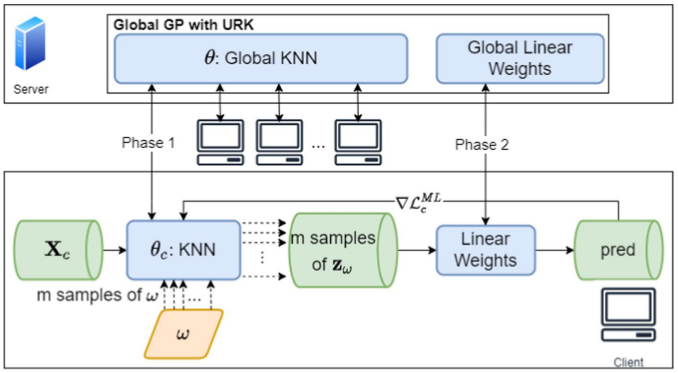

Fig. 1. Fig. 1 FedBNR learns a global federated GP in two phases: kernel learning with federated optimization and last layer updating with exact Bayesian inference 1 3 72 Page 8 of 36 Machine Learning (2026) 115:72 where sm ω (x) is the concatenation of m samples of zω(x). Equation 12 is the true underlying kernel we construct. Equation 13 is the Monte-Carlo approximation of Eq. 12, where the samples are the random features that URK produces. Furthermore, note Eω[zω(x)⊤zω(x’)] = Tr(Covω(zω(x), zω(x’))) + Eω[zω(x)]⊤Eω[zω(x’)] (14) Equation 14 follows from the definition of covariance (i.e., Cov(A, B) = E[AB] -E[A]E[B], where A, B are random variables). Tr refers to the trace, which is needed for random vectors. It is a decomposition of the true underlying kernel, which provides an intuitive understand ing of the construction where URK mainly captures the covariance between different ran dom basis functions. This aligns with the purpose of the kernel function.

Examples and Theoretical Guarantees Since g is an arbitrary function, we can assign it any DNN with any architecture. We call it a kernel neural network (KNN) since its weights, θ, are essentially the kernel hyperparameters. It is trivial to map a deep kernel defined by k($\phi\theta$(x), $\phi\theta$(x’)), where k is any conventional stationary kernel and $\phi\theta$ is any feature map ping function potentially represented by a deep neural network, into a URK construction: ω ∼p(ω), g(ω, x) = [sin(ω⊤$\phi\theta$(x)), cos(ω⊤$\phi\theta$(x))]⊤ (15) where p(ω) is the inverse Fourier transform of k. With m samples of ω, the error of approxi mating the true underlying kernel k(x, x’) drops exponentially: Theorem 3.1 ∀x, x’ ∈Rp, ∀p(ω) being a proper distribution over Rd’, and g : Rd’ × Rp -→Rd has the form of Eq. 15. With m ∈Z+ samples of ω, ∀t > 0 ( ) -mt2 Pr[|URKm ω,g(x, x’) -k(x, x’)| ≥t] ≤2 exp 8 URK can reconstruct not only conventional stationary kernels with a RFF mapping, but also non-stationary kernels beyond RFF’s capability. We give two example constructions and their corresponding convergence properties below. Full proofs are in Appendix D. The first example recovers a non-stationary kernel that induces infinite basis functions, like the Gaussian kernel.

Proposition 3.2 Let ω ∼N(0, I), g(ω, x) = exp(ω⊤x), then k(x, x’) = limm-→∞ URK m ω,g(x, x’) is a non-stationary kernel with infinite features.

The following theorem bounds the error of URK approximation for the construction in Prop. 3.2. Fenton-Wilkinson approximation (Fenton, 1960) is used to model the summation of log-normal distributions, and the error is bounded by the sum of two tail probabilities.

Theorem 3.3 Let ω and g have the form of Prop. 3.2. Suppose x, x’ ∈Rp satisfies ||x||2 2 ≤a, ||x’||2 2 ≤a’. With m ∈Z+ samples of ω, ∀t ∈(0, exp(a + a’)/2)), by FentonWilkinson approximation, 1 3 Page 9 of 36 72 Machine Learning (2026) 115:72 (exp (a + a’) -1)

IV. + 1

σs := log , E := exp((a + a’)/2) Pr[|URKm ω,g(x, x’) -k(x, x’)| ≥t] ≤1 -F (2 log(t + E) -(a + a’) + σ2 s 2σs) (2 log(E -t) -(a + a’) + σ2 s 2σs) + F F is the cumulative probability distribution function of the standard normal distribution.

The second example recovers a popular non-stationary kernel, the polynomial kernel.

Proposition 3.4 Let ~c, ~n ∈R be hyperparameters of the polynomial kernel k(x, x’) = (x⊤x’ + ~c)~n. Define ~p = [1 2 , 1 2p, 1 2p, · · · , 1 2p]⊤∈Rp+1, ω ∼Multi(~n,~p), the multinomial distribution. Define ~x = [√ 2~c, √2px⊤]⊤∈Rp+1, g(ω, x) = exp(ω⊤log~x), then k(x, x’) = limm-→∞URK m ω,g(x, x’) = (x⊤x’ + ~c)~n.

The following theorem bounds the error of URK approximation for the construction in Prop. 3.4, with Hoeffding’s inequality (Hoeffding, 1994) similarly to Thm. 3.1. Theorem 3.5 Let ω and g have the form of Prop. 3.4. Suppose entries of x, x’ ∈Rp + are bounded by [a, b]. With m ∈Z+ samples of ω, ∀t > 0, √ 2~c, √2pb} min{ √ v := max{ 2~c, √2pa} (-2mt2) Pr[|URKm ω,g(x, x’) -k(x, x’)| ≥t] ≤2 exp v4~n Without Randomness Note the deep features themselves already allow a valid BLR for mulation that is equivalent to a linear kernel. However, it is restricted to finite basis func tions as the same size of the second last layer and has the implicit assumption that the target is linear in the features. As the most basic kernel function, the linear kernel does not provide additional model capacity to the deep neural network. Previously, deep kernel learning algo rithms (Wilson et al., 2016a, 2016b) apply the Gaussian kernel, which induces infinite basis functions, to deep features and show they effectively improve the model accuracy compared to using deep features only. The rationale is straightforward: adding extra basis functions to a model never worsens its training accuracy, and GPs are resilient to over-fitting. Adding randomness with the URK construction follows the same idea. It further extends the model capacity of the KNN by allowing an effective approximation to infinite kernels. In Appen dix C.1, we report an extra ablation study that shows empirically the URK approximation indeed improves the accuracy in most cases compared to deep features only. Learn the Distribution URK is meant to be a generic

definition that enables any archi tecture g and any distribution for ω. Whether a specific instantiation is preferable is highly problem dependent. Nevertheless, we present an architecture that utilizes the flexibility of deep learning and let the model learn automatically a distribution for ω that best fits its data. The architecture is derived in alignment with deep kernel learning proposed by Wilson et al. 1 3 72 Page 10 of 36 Machine Learning (2026) 115:72 (2016b). As illustrated in the left half of Fig. 2, URK can recover any DNN combined with a conventional stationary kernel, using the architecture mentioned. p(ω) of popular station ary kernels are reported in RFF (Rahimi & Recht, 2007). For example, the Gaussian kernel simply corresponds to ω ∼N(0, I), the standard normal distribution. Furthermore, we can exploit the flexibility of URK and define a distribution shifter h that transforms ω. We can start from a standard normal distribution, which is very easy to sample from, and send the samples through h to simulate a much more complex distribution with minimal computa tional resources required. We choose h such that the identity function is in its hypothesis set, which can be achieved by adding (potentially linearly transformed) ω to the output of any architecture, similarly to the residual network (He et al., 2016). Then, we can learn a distri bution for ω that is at least as good as the Gaussian kernel in deep kernel learning.

3.2 FedBNR: Federated Learning of Global GPs with URK With URK, we can now approximate infinite deep kernels with finite random features and work in the feature space. The training procedure of our proposed algorithm FedBNR can be divided into two phases, as illustrated in Fig. 1. In the first phase, we train the KNN by optimization methods. In the second phase, we calculate the weights of the last linear layer that maps the random features to the output space, by a closed-form formula inferred from Eq. 11. The pseudocode of FedBNR is summarized in Algorithm 1. In phase 1, we follow a standard training procedure under the FL framework. We assume there is a central server holding the shared global KNN weights θ and global hyperparam eters σ, λ that denote the noise level and the prior variance respectively. We assume there is a set of clients c ∈S holding local KNN weights θc, local hyperparameters σc, λc, local inputs Xc, and local targets yc. We use $\phi\theta(\omega$, x) to denote the resulting features of x with a specific set of random samples ω with respect to θ. ω is omitted for simplicity when it is not fixed. In the beginning of each aggregation round, the server sends a copy of aggregated or initialized θ, σ, λ to all the clients. All the clients c ∈S first update their local model for

V. = -LOG PRBLR(YC|$\phi\theta$C(XC); σC, λC)

LML

c ¯ wc. (16) cI||Ac|)||yc -Φc ¯ wc||2 + λ-2 ¯ w⊤ = nc log σ2 c + log(|λ2 One commonly used method for aggregation is the FedAvg (McMahan et al., 2017) heu ristic, where the new global model parameters are assigned the average of all client model parameters θ = ∑ c∈S θc/|S|. However, multiple works (Karimireddy et al., 2020; Shoham et al., 2019; Mohri et al., 2019) have pointed out the quality and the convergence rate

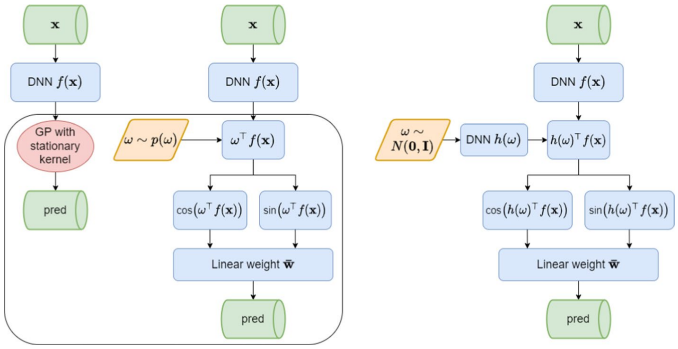

Fig. 2. Fig. 2 From left to right: a standard deep kernel learning algorithm with conventional stationary kernel GPs; the corresponding URK architecture; a more general architecture enabled by URK for convenient latent stationary kernel learning 1 3 Page 11 of 36 72 Machine Learning (2026) 115:72 a fixed number of iterations. Then they send θc, σc, and λc back to the server for aggrega tion and start another aggregation round. The local loss function is the negative local log marginal likelihood:

of this heuristic can suffer from non-i.i.d. clients. To account for this, we propose to adapt kernel knowledge distillation (He & Ozay, 2022) to aggregate the KNNs. We assume the server holds a relatively small dataset Xkd, ykd and tries to minimize the following knowledge distillation loss with respect to this dataset: ∑

LKD = LML kd + α ∗MSE($\phi\theta$(Xkd)⊤$\phi\theta$(Xkd), $\phi\theta$c(Xkd)⊤$\phi\theta$c(Xkd)/|S|). (17) c∈S here, LML kd is the log marginal likelihood loss LML with respect to the knowledge distilla tion dataset Xkd, ykd. α is a common hyperparameter in knowledge distillation methods to adjust the ratio between the log marginal likelihood loss and the mean squared error (MSE) loss. The MSE loss forces the global kernel $\phi\theta$(Xkd)⊤$\phi\theta$(Xkd) to simulate the mean of all client kernels, namely ∑ c∈S $\phi\theta$c(Xkd)⊤$\phi\theta$c(Xkd)/|S|, which is akin to concatenating all the features of the client kernels. Ideally, if the global kernel successfully learns to do so, it should not perform worse on any of the clients, while the FedAvg heuristic has no similar guarantees.

In phase 2, we fix the kernel hyperparameters and learn ¯w in an exact way as if all client data is centralized. First, note that we can decompose ¯w as follows: ¯w = σ-2(σ-2ΦΦ⊤+ λ-2I)-1Φy

(18) = σ-2(σ-2 ∑ c∈S Φc Φ⊤ c + λ-2I)-1 ∑ c∈S (Φcyc)

(19) Φc = $\phi\theta$(Xc) denotes the random features of local inputs extracted by the global KNN. The server first broadcasts the global model to all the clients and asks them to return the scatter matrices ΦcΦ⊤ c and the vectors Φcyc. Then, the server simply takes the summation of cli ent messages, calculates ¯w according to Eq. 19, and broadcasts the whole model again to all the clients. Note that phase 2 only needs to be done once after kernel learning to achieve the global model. There is no need to iterate between the two phases.

In phase 1, each local update costs O(nc(md)2 + (md)3) time to compute and inverse Ac and O((md)2) space to store Ac. This is equivalent to the complexity of BLR models with random feature approximations. It is more scalable than

working with the kernel K directly, which costs O(n3 c) time and O(n2 c) space when md ≪nc. Each client shares a message of size |θ| with the server. FedAvg aggregation then costs O(|θ|) time and O(|θ|) space, while 1 3 72 Page 12 of 36 Machine Learning (2026) 115:72

KD aggregation costs O(nkd(md)2 + (md)3 + n2 kdmd) time and O((md)2 + n2 kd) space, where nkd is the size of the knowledge distillation dataset. In phase 2, each client calculates the scatter matrices and the vectors, costing O(nc(md)2) time and O((md)2) space. Each client shares a message of size (md)2 with the server. Finally, learning the global posterior costs O((md)3) time and O((md)2) space, similarly to phase 1. Note that if nid inducing points are used to approximate the full kernel (e.g. in pFedGP), O(n3 id) time and O(n2 id) space is needed similarly.

Algorithm 1 Federated Bayesian neural regression (θ: the global KNN, σ: noise level, λ: prior covariance of the linear layer, ω: a set of random numbers, ζ: local learning step, ζ': knowledge distillation step)

3.3 Enhance Protection with DP We claim that FedBNR protects the privacy of clients at least as well as FedAvg and other FL algorithms that send client models to the server. In phase 1, the aggregation only requires client model parameters. In phase 2, we send two messages outside each client: ΦcΦ⊤ c and Φcyc. These matrices and vectors (of size m × m and m) are also highly compressed summaries of the raw data, which prevents immediate privacy invasions as discussed in Appendix A. We note however that FL with compressed messages does not prevent privacy attacks (Boenisch et al., 2021; Mothukuri et al., 2021), it simply makes it harder to infer the underlying data. To provide formal privacy guarantees, federated learning needs to be com bined with formal mechanisms such as differential privacy (DP) (Wei et al., 2020; Triastcyn & Faltings, 2019) or secure multi-party computation (MPC) (Truex et al., 2019; Byrd & Polychroniadou, 2020; Li et al., 2020).

1 3 Page 13 of 36 72 Machine Learning (2026) 115:72 Definition 1 A mechanism M satisfies (ϵ, δ)-DP if for any two datasets D, D' that differ by only one record (i.e. |(D -D') ∪(D' -D)| = 1, noted as D ∼D'), and for any possible output O ∈Range(M), (Pr[M(D) = O] [ )] < δ. Pr O∼M(D) log > ϵ Pr[M(D') = O] The idea of (ϵ, δ)-DP is to add enough noise to the shared information so that with high probability (0 ≤δ < 1), the distribution of the shared message does not change much when the raw dataset is modified by one record (controlled by ϵ > 0). Thus, one cannot effectively tell if any record is included in the dataset. (ϵ, δ)-DP models the information leakage of the raw dataset with the inner log ratio of Def. 1, and guarantees that it is bounded by ϵ, the pri vacy budget, with probability δ. Empirically, people set ϵ ≤1 for strong privacy protection, while ϵ ≥10 is deemed weak. We will use the following mechanism: Theorem 3.6 (e.g. (Kerkouche et al., 2021)). A Gaussian Mechanism GM is (ϵ, δ)-DP with information revealed f: GM(D) = f (D) + N(0, σ2 I), where σ = L2(f) √

2 ln(1.25/δ)/ϵ. L2(f) calculates the L2 sensitivity of function f as follows. L2(f) = max D,D' ||f(D) -f(D')||2, D ∼D'. (20) We now extend FedBNR to be differentially private with the Gaussian mecha nism. In phase 1, the L2 sensitivity of the kernel hyperparameters is the L2 gradi ent clipping constant times the number of local update rounds times the learning rate ζ. In phase 2, we send [ΦcΦ⊤ c , Φcyc, y⊤ c yc] instead and its L2 sensitivity is √ 32 + 16/m + 8b2 + b4, where b := max(y) -min(y), assuming the URK construction is g(ω, x) = [sin(ω⊤ϕNN(x)), cos(ω⊤ϕNN(x))]⊤. See details in Appendix A. Then, notice the following matrix is PSD without DP: [Φc y⊤ c] [[ΦcΦ⊤ c Φcyc (Φcyc)⊤ y⊤] . (21)] = Φ⊤ c yc c yc After adding Gaussian noise, we project the matrix back to the nearest PSD matrix by set ting its negative eigenvalues to 0. Then we perform regular BLR with projected ΦcΦ⊤ c and Φcyc. 4 Experiments 4.1 Synthetic Experiments We designed the following synthetic experiments to show local GP models cannot handle generalization or new clients. We first decide a true underlying function, sample 200 points uniformly from the range [-5, 5], and add Gaussian noise with σ = 0.5. We then learn the kernel hyperparameters in a centralized fashion to eliminate any impact from imperfect ker 1 3 72 Page 14 of 36 Machine Learning (2026) 115:72 nels later on. We assign the first 100 points in range [-5, 0] to client 1, and the rest to client 2. We train pFedGP and FedBNR with the learned kernel hyperparameters fixed and query for prediction in range [-5, 5]. The top left graph of Fig. 3 shows the result of pFedGP, and the top right one shows the result of FedBNR. The blue curve shows the true underlying function, and the green curve shows the predictions. The light red area is a 95% confidence interval based on variance. The purple dots are the data leveraged by each model for the last layer. Since both algorithms learn hyperparameters centrally, the only difference lies in whether they leverage data of both clients for the predictive model. As shown in the pFedGP graph, the personalized model of client 1 only sees its own data, so it does not generalize well to the range of client 2, while the global model learned by FedBNR leverages all the data and generalizes to the full range. Next, we introduce two new clients into the system. The first client holds data uniformly sampled from the range [-5, 5], and the second from the range [5, 15]. We again fix the kernel hyperparameters learned centrally and train pFedGP, FedLoc, and FedBNR on these new clients separately. For testing, we still query the range [-5, 5]. As shown in the bot tom graphs of Fig. 3. The quality of prediction of the pFedGP model is massively impacted when there are few points in the first case, and for FedBNR the loss remains approximately a straight line. For FedLoc, the loss is also impacted due to zero-out effects of non-over lapping client models. Even worse, when training data and testing data are not in the same range for the second new client, the MSE loss of both pFedGP and FedLoc never drops back to the level of previous clients. __IMAGE__intermediate/figures/fig_3.jpeg__PAGE_13__Y_322__ Fig. 3 Top graphs: (i) left: prediction of pFedGP trained on two non-overlapping clients; (ii) right: global prediction of FedBNR. Blue curve: underlying truth; green curve: point

estimation; red range: 95% con fidence interval; purple dots: training data leveraged by the model. Bottom graphs: MSE of predictions tested on the same range v.s. number of training data from a new client; (i) left: client with training range [-5, 5]; (ii) right: client with training range [5, 15] 1 3 Page 15 of 36 72 Machine Learning (2026) 115:72 4.2 Real World Regression Tasks To show our algorithm achieves better generalization performance in terms of both point and uncertainty estimation, we conducted comprehensive experiments on 5 real world regression tasks under 10 cases, including Skillcraft (Thompson et al., 2013), SML (Zamora-Martinez et al., 2014), Parkinsons (Tsanas et al., 2009), Bike (VE & Cho, 2020), and CCPP (Tüfekci, 2014). We note while a common challenge for GPs is to scale to large datasets, in FL, the challenge is the opposite: how to combine models trained on small datasets. When the datasets of each client become sufficiently large, then there is no need to combine models nor do FL. Hence, the experiments focus on reducing the amount of data per client while increasing the number of clients. The scalability of FedBNR is trivially guaranteed by the use of random features. Results of two variants are reported: (i) FedBNR that performs the FedAvg heuristic at aggregation; and (ii) FedBNR-KD that performs the knowledge distillation method at aggregation. Two groups of baselines are compared to our method for RMSE error: (i) ablation study that includes local+local, local+global, avg+local, and kd+global, in the format of kernel learning method + last linear weight learning method, where we remove the global aggregation of either Phase 1 or Phase 2 from our methods; (ii) previous works that include (1) FedAvg (McMahan et al., 2017), a standard non-Bayesian FL algorithm that has a global model; (2) FedProx (Li et al., 2020), a non-Bayesian FL algorithm that adds a proximal loss to FedAvg to prevent client models from getting too far from the global model; (3) pFedGP (Achituve et al., 2021), a Bayesian PFL method that learns a local GP with a shared deep kernel for each client using FedAvg; (4) FedLoc (Yin et al., 2020), a Bayesian FL method that directly applies distributed GP methods (Xie et al., 2019) without deep kernel learning; (5) DEC-apx-GP (Kontoudis & Stilwell, 2023), a decentralized Bayesian PFL method that learns a local GP with a shared separable squared exponential kernel for each client using proximal ADMM; and (6) DEC-gapx-GP (Kon toudis & Stilwell, 2024), an extension of DEC-apx-GP that shares an additional dataset with all clients as extra training data. We also compare to pFedGP, FedLoc, DEC-apx, and DECgapx for calibration errors since they are Bayesian models that have a notion of confidence. Besides, we report results of a centralized GP equipped with URK as a casual reference of the testing error lower bound. We used fully connected neural networks to extract firststep features for FedAvg, FedProx, and pFedGP. We used Gaussian kernels for the GPs in pFedGP and FedLoc. For fairness, KNNs used by our methods have similar architectures to the combined kernel of pFedGP. FITC (Snelson & Ghahramani, 2005) approximations are implemented for pFedGP and FedLoc as described in pFedGP. We used 50 random samples for KNN and 50 inducing points for pFedGP and FedLoc. Further details are in Appendix B. Each dataset is uniformly divided into training, testing, and validation sets globally with a ratio of 8:1:1. The training data is sorted by the feature that has the largest absolute cor relation coefficient with the output, and divided into multiple chunks. Each client randomly takes two chunks so their data distributions are heterogeneous. The larger the absolute cor relation coefficient is, the more significant the distribution skew is. Before training, we tune hyperparameters that cannot be learned by gradient descent with grid searching on the validation set. Specially, FedBNR-KD uses 80% of the validation set for knowledge distil lation, and the rest 20% for validation. All datasets are ran for 50 local epochs times at most 100 aggregation rounds in full batches. Validation and testing errors are recorded for each aggregation round. For methods that contain only local models, these errors are defined as 1 3 72 Page 16 of 36 Machine Learning (2026) 115:72 the mean error of all local models with respect to the testing/validation set. If the validation error has not improved for 5 rounds, the training process is terminated. We report the average minimum testing RMSE for 10 random seeds for each case in Table 1, which assesses the point estimation of different models. We also measured the statistical significance of the results compared to FedBNR with one-tailed Wilcoxon signedrank tests (Wilcoxon, 1992). We report expected calibration errors (ECE) in Table 2 and per form the Wilcoxon test compared to FedBNR-KD. ECE is a calibration metric that assesses the uncertainty estimation. Further metrics are included in Appendix C. The results show: (i) in terms of RMSE, our methods are statistically better than the other models in most of the cases, especially when the training set at each client is significantly smaller than the whole set; (ii) compared with the ablation study methods, our methods always perform better, so the global aggregation at both phases is essential; (iii) FedLoc without deep kernel has a much smaller model capacity; (iv) FedBNR-KD only outperforms FedBNR in 40% of cases in terms of RMSE, which is probably due to the small size of data (8% of all) used for knowledge distillation. However, it is clearly more stable when client heterogeneity gets worse as the number of clients increases. It is also better calibrated in most cases. 4.3 Personalization We conducted experiments with the personalization setting to show that the global GP posterior benefits uncertainty estimation even in the personalization setting. FedBNR can be naturally extended to the personalization setting by fine-tuning the global model. During fine-tuning, we minimize LF T c = ||yc -$\phi\theta$c(Xc)wc||2 2 + λ-2 c ||wc||2 2, the local regu Table 1 Real world regression datasets, RMSE reported for the generalization setting. ⇑ and ↑ denote sig nificantly worse results with p < 0.01 and p < 0.05 respectively; ⇓ and ↓ denote significantly better results similarly

	Skillcraft	SML	Parkinsons	Bike	CCPP
Train/test size	2670/334	3309/414	4700/587	7008/876	7654/957
corr-coef	-0.660	0.783	0.410	0.539	-0.948
#clients	10 100	10 100	10 100	10 100	10 100

10 100 10 100 Central GP 0.95 0.21 3.58 0.37 4.02

Local+local 1.26⇑ 1.48⇑ 1.49⇑ 2.32⇑ 10.9⇑ 10.6⇑ 0.79⇑ 0.92⇑ 14.6⇑ 19.3⇑ Local+global 1.08⇑ 1.22⇑ 1.00⇑ 1.65⇑ 6.42⇑ 7.42⇑ 0.59⇑ 0.73⇑ 5.62⇑ 7.03⇑ Avg+local 1.05⇑ 1.06⇑ 0.61⇑ 0.81⇑ 9.84⇑ 8.80⇑ 0.45⇑ 0.54⇑ 8.32⇑ 13.4⇑ kd+local 1.04⇑ 1.28⇑ 0.86⇑ 1.34⇑ 6.15⇑ 6.97⇑ 0.51⇑ 0.61⇑ 10.0⇑ 17.1⇑ FedAvg 1.00↑ 1.03⇑ 0.36⇑ 0.71⇑ 6.83⇑ 7.38⇑ 0.38↓ 0.42 4.45 4.44 FedProx 0.98 1.05⇑ 0.34↑ 0.62⇑ 6.32⇑ 7.38⇑ 0.39 0.42 4.43 4.47 pFedGP 0.99↑ 1.15⇑ 0.75⇑ 1.34⇑ 9.36⇑ 8.91⇑ 0.45⇑ 0.46⇑ 14.2⇑ 18.2⇑ FedLoc 1.08⇑ 4.15⇑ 2.83⇑ 5.43⇑ 8.40⇑ 11.5⇑ 0.64⇑ 0.75⇑ 20.6⇑ 44.1⇑ DEC-apx 1.10⇑ 4.34⇑ 1.90⇑ 4.93⇑ 8.59⇑ 9.21⇑ 0.76⇑ 0.88⇑ 7.64⇑ 11.06⇑ DEC-gapx 0.94⇓ 4.31⇑ 1.41⇑ 3.57⇑ 8.07⇑ 8.26⇑ 0.55⇑ 0.75⇑ 6.26⇑ 9.12⇑ ours FedBNR 0.98 0.97 0.25 0.44 3.10 5.42 0.39 0.42 4.40 4.51 FedBNR-KD 0.96⇓ 0.98↑ 0.55⇑ 0.55⇑ 4.58⇑ 4.67↓ 0.43⇑ 0.48⇑ 4.38 4.38↓ 1 3 Page 17 of 36 72 Machine Learning (2026) 115:72 Table 2 ECE reported for the generalization setting. ⇑, ↑, ⇓, and ↓ denote significantly worse or better results similarly to Table 1

Skillcraft SML Parkinsons Bike CCPP #Clients 10 100 10 100 10 100 10 100 10 100 Central GP 0.02 0.09 0.27 0.11 0.24

pFedGP 0.43⇑ 0.38⇑ 0.45⇑ 0.45⇑ 0.49⇑ 0.48⇑ 0.42⇑ 0.41⇑ 0.49⇑ 0.49⇑ FedLoc 0.32⇑ 0.44⇑ 0.12⇓ 0.27⇑ 0.32⇑ 0.43⇑ 0.26⇑ 0.16⇑ 0.23⇓ 0.50⇑ DEC-apx 0.50⇑ 0.46⇑ 0.50⇑ 0.50⇑ 0.50⇑ 0.50⇑ 0.19⇑ 0.14⇑ 0.42⇑ 0.38⇑ DEC-gapx 0.50⇑ 0.36⇑ 0.50⇑ 0.50⇑ 0.50⇑ 0.50⇑ 0.28⇑ 0.19⇑ 0.47⇑ 0.42⇑ ours FedBNR 0.05 0.20⇑ 0.39⇑ 0.37⇑ 0.36↑ 0.40⇑ 0.07 0.04⇑ 0.24⇓ 0.20⇓ FedBNR-KD 0.05 0.06 0.20 0.21 0.29 0.30 0.08 0.09 0.30 0.31 larized least square loss. Note the weights in the last linear layer are also updated by gradient descent. Varying the number of epochs of fine-tuning offers a controllable trade-off between generalization and personalization. Training datasets are distributed to clients as before for heterogeneity, but testing and validation sets are sampled with the same distribution as local training datasets, with a ratio of 14:3:3. We report the RMSE and ECE in Table 3, where pxADMM (Xie et al., 2019) is a distributed GP hyperparameter learning algorithm used by FedLoc and we extend it with deep kernels. The results show that FedBNR achieves compa rable accuracy and better calibration than other methods, indicating a global posterior helps calibrate local models. 4.4 Differential Privacy We conducted experiments for FedBNR with DP to show the trade-off between model utility and the privacy budget ϵ. Experiment details are similar to Sect. 4.2. The results of the Skillcraft and SML datasets are summarized in Fig. 4. As the graph shows, the RMSE loss of fully differentially private FedBNR quickly drops back to the RMSE of vanilla Fed Log, when $\epsilon \geq 0.1$. The DP noise affects the model less when there are more clients, since in Phase 1 the messages are averaged. Extra results are in Appendix C. 5 Conclusion In this work, we proposed FedBNR, a novel algorithm that learns a global federated GP that provides better generalization and uncertainty estimation. FedBNR learns a deep kernel under feature mapping, and shares scatter matrices instead of direct features to achieve the exact global optimum of the last layer. It shows empirically statistically significant improve ments in terms of point and uncertainty estimation compared to other federated GPs. One limitation of FedBNR is that the decomposition of global posterior learning with random features is restricted to regression tasks, and it is non-trivial to extend to the GP classifica tion case where Laplacian approximation is needed. 1 3 72 Page 18 of 36 Machine Learning (2026) 115:72 Table 3 RMSE (top half) and ECE (bottom half) reported for the personalization setting. ⇑ and ↑ denote significantly worse results with $p < 0.01$ and $p < 0.05$ respectively; ⇓ and ↓ denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP 10 100 10 100 10 100 10 100 10 100 pFedGP 1.00⇓ 1.08⇓ 0.31⇑ 0.64 2.97⇑ 3.66⇑ 0.47 0.44⇓ 4.72⇑ 12.66⇑

pxADMM 0.99⇓ 1.16⇓ 2.11⇑ 1.75⇑ 6.99⇑ 3.73⇑ 0.56⇑ 0.50⇓ 10.21⇑14.60⇑

DEC-apx 1.10 4.19⇑ 1.59⇑ 3.49⇑ 3.56⇑ 3.32⇑ 0.57⇑ 0.71⇑ 4.40⇑ 4.92⇑ DEC-gapx 0.96⇓ 2.91⇑ 1.15⇑ 3.30⇑ 4.49⇑ 5.83⇑ 0.54⇑ 0.64⇑ 4.40⇑ 5.74⇑ ours FedBNR 1.12 1.25 0.25 0.62 1.39 1.75 0.47 0.57 3.89 4.77 FedBNR-KD 1.12 1.22⇓ 0.38⇑ 0.66 1.78⇑ 2.13⇑ 0.49⇑ 0.63⇑ 3.95⇑ 4.74 #Clients Skillcraft SML Parkinsons Bike CCPP 10 100 10 100 10 100 10 100 10 100 pFedGP 0.42⇑ 0.35⇑ 0.45⇑ 0.44⇑ 0.45⇑ 0.44⇑ 0.42⇑ 0.39⇑ 0.49⇑ 0.49⇑ pxADMM 0.42⇑ 0.34⇑ 0.50⇑ 0.45⇑ 0.49⇑ 0.44⇑ 0.20⇑ 0.21⇑ 0.50⇑ 0.49⇑ DEC-apx 0.50⇑ 0.50⇑ 0.50⇑ 0.50⇑ 0.50⇑ 0.50⇑ 0.27⇑ 0.21⇑ 0.27⇑ 0.43⇑ DEC-gapx 0.50⇑ 0.50⇑ 0.50⇑ 0.50⇑ 0.45⇑ 0.50⇑ 0.28⇑ 0.24⇑ 0.20⇓ 0.12⇓ ours FedBNR 0.01 0.04 0.24 0.28 0.04 0.04 0.11 0.08 0.25 0.33 FedBNR-KD 0.01 0.03⇓ 0.13⇓ 0.17⇓ 0.06⇑ 0.09⇑ 0.11 0.07 0.29⇑ 0.33 __IMAGE__intermediate/figures/fig_4.jpeg__PAGE_17__Y_327__ Fig. 4 FedBNR+DP results. Left: Skillcraft dataset; right: SML dataset. Phase 1 DP: add DP protection to θc only; full DP: add DP protection to all messages Appendix A: Privacy Risks Review: Why DL or Feature Sharing is Not Private We provide a more detailed discussion in this section to support our claim that directly using previous works about distributed GPs is not safe regarding privacy, and that global federated GP is challenging itself. In terms of privacy, the ultimate goal of FL is to prevent Xc and yc from being (easily) recovered by any other entity, including the server, than their original owner c. We first analyze common risky messages by showing how to recover the raw data 1 3 Page 19 of 36 72 Machine Learning (2026) 115:72 Table 4 Privacy leakage of dif ferent GP algorithms

Technique Risky message Prob lematic term Leaked info Xc or Φc –Xc, yc Inducing points U or Uc –⊂Xc k(Xc, U) –yc FedAvg (2017) N/A N/A N/A pFedGP (2021) N/A N/A N/A FedBNR (ours) N/A N/A N/A Centralized GP Xc/Φc K, k(x⋆, X) Xc, yc Risky message: messages that leak privacy but unavoidable for global prediction. Problematic term: terms that the risky messages try to compute. Leaked info: private information that can be recovered by adversaries with the risky messages. ⊂ means "part of"

FITC (2005) U, k(Xc, U) k(X, U) ⊂Xc, yc pFedGP + Global Predictor U, k(Xc, U) k(X, U) ⊂Xc, yc Distributed

GP (2015) x⋆/Φ⋆ or Xc/Φc k(x⋆, Xc)Xc, yc or X∗, y∗ Distr. Variational GPs (2014; 2016) U k(U, ·) ⊂Xc

SPAHM GP (2019) U k(U, ·) ⊂Xc with them. We then discuss privacy leakage of different GP algorithms and summarize them in Table 4. We will now discuss messages and algorithms listed in the table one by one.

Risky Messages

Features Xc or Φc Without loss of generality, we can denote the prediction model by y = f(ϕ(x)), where ϕ(Xc) = Φc. Then the adversaries can simply compute yc = f(Φc) to recover the targets of clients. This further extends to sharing x⋆ or Φ⋆, as the testing points and their corresponding targets can also be confidential.

Inducing Points U or Uc There are different ways of choosing inducing points. One way is to select a subset of the training data X, and this clearly leaks raw data. Another way is to optimize them as hyperparameters. However, Smith et al. (2021) pointed out that inducing points optimized in this way often end up in regions of low data density and may corre spond to outliers. In this case, there is a high risk that some inducing points will correspond directly to real data. They proposed to use differential privacy to protect the output, but the input is left unprotected. Even in the case where inducing points lie in different spaces than the original input, it is still possible to partially recover real yc with the same argument as Φc. Thus we want to refrain from using inducing points.

Cross Kernel Matrix k(Xc, U) In general, sharing k(Xc, U) does not reveal Xc imme diately since the kernel function k tends to be non-invertible. However, in many algorithms that use inducing points, k(Xc, U) serves as sufficient statistics to predict yc. For example, for FITC (Snelson & Ghahramani, 2005), the predictive mean is given by: f(x⋆) = k(x⋆, U)Σk(U, X)Λ-1y, where Σ = (k(U, U) + k(U, X)Λ-1k(X, U))-1 Λ = diag[K -k(X, U)k(U, U)-1k(U, X)] 1 3 72 Page 20 of 36 Machine Learning (2026) 115:72 Then the adversaries can again retrieve yc = k(Xc, U)Σk(U, X)Λ-1y without knowing Xc.

Algorithms

FedAvg FedAvg shares no risky messages listed before. It shares model parameters only. pFedGP Similar to FedAvg, pFedGP only shares model hyperparameters. Any prediction that requires K, k(x⋆, X) resorts to a local version Kc, k(x⋆, Xc). FedBNR In phase 1, we share hyperparameters like FedAvg and pFedGP, which is safe. In phase 2, we share hyperparameters and three other terms back and forth. No risky mes sages are shared.

\textbullet message1 := ΦcΦ⊤ c : The outer product (known as the scatter matrix) is a non-invertible function of Φc, and our model requires at least Φc to predict yc. \textbullet message2 := Φcyc: This alone poses an under-constrained system of equations that has m equations and (m + 1)nc variables, so Φc, yc is non-identifiable. \textbullet message3 := ¯w = σ-2(σ-2 ∑ c∈S ΦcΦ⊤ c + λ-2I)-1 ∑ c∈S(Φcyc): This alone poses an under-constrained system of equations that has m equations and (m + 1)n variables, so Φ, y is non-identifiable.

In total, these three messages pose a system of equations that has m(m + 1)/2 + 2m = 0.5m2 + 2.5m equations and (m + 1)n variables (i.e. Φ, y). Since m ≪n, meaning the number of random samples is much smaller than the size of training data, the system has multiple solutions and the true values of Φ, y are non-iden tifiable. Similarly, the first two messages pose a (local) system of equations that has m(m + 1)/2 + m = 0.5m2 + 1.5m equations and (m + 1)nc variables (i.e. Φc, yc). As long as 1 < m ≤nc, the variables are still non-identifiable.

Centralized GP The predictive mean is given by: f(x⋆) = k(x⋆, X)(K + σ2I)-1y Thus, even if we can learn the hyperparameters in a FL fashion without privacy leakage, we have to share Xc/Φc to calculate K and k(x⋆, X).

FITC FITC has to share k(Xc, U) to calculate the term k(X, U) in Σ. See FITC in the paragraph of Cross kernel matrix k(Xc, U) for details. Also, inducing points U have to be shared with all clients.

pFedGP + Global Predictor Following the arguments with centralized GP and induc ing point approximation, pFedGP cannot generalize to a global predictor without sharing Xc/Φc (without approximation) or U and k(Xc, U) (with the FITC approximation they adapt).

Distributed GP Deisenroth and Ng (2015) introduces a collection of works that uses the Product of Experts (PoE) model to approximate the global GP with local GPs. In the most basic setting, the global predictive mean and variance are given by: 1 3 Page 21 of 36 72 Machine Learning (2026) 115:72 μ(x⋆) = σ2(x⋆) ∑ c∈S σ-2 c (x⋆)μc(x⋆) ∑ σ-2(x⋆) = σ-2 c (x⋆) c∈S where μc(x⋆) and σ2 c(x⋆) are the local predictive mean and variance. Then, to calculate these two terms, we either have to share x⋆/Φ⋆ or collect all the client experts for the tester to perform global prediction locally, where a client expert is a stand-alone GP predictive model with local training data. However, collecting client experts requires sharing Xc/Φc, by the formulas of predictive mean and variance of GP. Note all the other advanced methods proposed by Deisenroth and Ng (2015) have similar structures and thus are not safe. Distributed Variational GPs These works are also based on inducing points, which may leak real data when clients have low data density regions, which is common for high dimen sional data.

SPAHM GP This paper did not specify technical details about the GP predictor they learned, so it remains unclear if there are other risky messages sent during the process, but they mentioned they used inducing points to learn GPs, which needs to be avoided in FL. A Differential Privacy (DP) Argument In this section, we show why sharing kernels is less private than sharing scatter matrices by examining how much noise is needed with DP. The amount of noise needed with DP significantly increases when the information released has size that is dependent of the size of data, when the released information con tains non-constant interaction terms between records. This often trashes the utility of the trained model, suggesting that such messages themselves contain a lot information about the raw data and are prone to privacy bleaching. As a concrete comparison, we can calculate the

sensitivity of fk(X) = Φ(X)⊤Φ(X), the n × n sized kernel, and fs(X) = Φ(X)Φ(X)⊤, the m × m sized scatter matrix. For simplicity, use Φ(X) = sin(X), ∀X ∈Rn×m so that its sensitivity is bounded. W.l.o.g, let X' = X[0 : n -1, :] be the first n -1 records of X. We pad fk(X') with 0's so that fk(X) and fk(X') are then both of size n × n. Then, 1 3 72 Page 22 of 36 Machine Learning (2026) 115:72 k(x1, x1) · · · k(x1, xn-1) k(x1, xn) k(x2, x1) · · · k(x2, xn-1) k(x2, xn) ... ... k(xn-1, x1) · · · k(xn-1, xn-1) k(xn-1, xn) k(xn, x1) · · · k(xn, xn-1) k(xn, xn) fk(X) -fk(X') = k(x1, x1) · · · k(x1, xn-1) 0 k(x2, x1) · · · k(x2, xn-1) 0 ... ... k(xn-1, x1) · · · k(xn-1, xn-1) 0 0 · · · 0 0 - 0 · · · 0 k(x1, xn) 0 · · · 0 k(x2, xn) ... ... 0 · · · 0 k(xn-1, xn) k(xn, x1) · · · k(xn, xn-1) k(xn, xn) = where k(xi, xj) = sin(xi)⊤sin(xj). We flatten the above matrix to a vector and cal culate the L2 sensitivity. There are 2n -1 non-zero terms in the above result. Note k(xi, xj) ≤m, ∀i, j. By definition, L2(fk) = max X,X' ||flatten(fk(X) -fk(X'))||2 = ||[0, · · · , 0 (n-1)2 terms]||2 , m, · · · , m

2n-1 terms √ = 2n -1m On the other hand, for scatter matrices, n-1 n fs(X) -fs(X') = i=1 sin(xi) sin(xi)⊤- i=1 sin(xi) sin(xi)⊤ = sin(xn) sin(xn)⊤ L2(fs) = max X,X' ||flatten(fs(X) -fs(X'))||2 = max x ||flatten(sin(x) sin(x)⊤)||2 = ||[1, · · · , 1 m2 terms]||2 = m Since L2(fk) grows with the size of data, the magnitude of noise needed to make fk differ entially private also grows to infinity, while L2(fs) stays a small constant m ≪√2n -1m. 1 3 Page 23 of 36 72 Machine Learning (2026) 115:72

Variance Adversarial Attack

We acknowledge that uncertainty quantification could pose potential negative consequences for privacy. For example, it is well-known that with stationary kernels, the variance of the GP predictive posterior tends to form local minima at places where training points locate, since by definition the variance of stationary kernels depends on the distance to the training points. In this case, it is possible to at least retrieve the target y by searching for the local minima of the variance. However, this property does not hold true in general when we work with non-stationary deep kernels, where the variance depends on the input locations. Figure 5 shows an example of FedBNR where the variance adversarial attack fails. The left graph shows the "real" training data we wish to protect, and the corresponding predic tive posterior learned by FedBNR with a deep kernel. The right graph shows the amount of variance for the prediction in the left graph. We can see the local minima of predictive variance are reasonably far away from training data, and thus the variance adversarial attack described above fails to retrieve any training data.

FedBNR+DP: Sensitivities

To use the Gaussian mechanism to protect FedBNR, we need to add noise to all the mes sages sent by the clients. In the first phase, kernel hyperparameters are revealed. In the second phase, ΦcΦ⊤ c , Φcyc, and y⊤ c yc are revealed. The L2 sensitivity of each function is as follows: \textbullet In phase 1, L2 sensitivity of the hyperpameters is the L2 gradient clipping constant times the number of local update rounds times the learning rate ζ. Since we take the average during aggregation, the actual sensitivity of the mean has an extra factor of 1/√nS, where nS is the number of clients. This means the more clients there are in the system, the less perturbation the mean exhibits. This factor further decreases to 1/nS if we adapt secure multi-party computation and add noise directly to the summation of all the messages globally, instead of adding noise to messages at clients. \textbullet In phase 2, suppose we use the latent stationary architecture (sin, cos activation functions normalized by √m), then each entry of Φc can change by up to 2/√m. y is in general

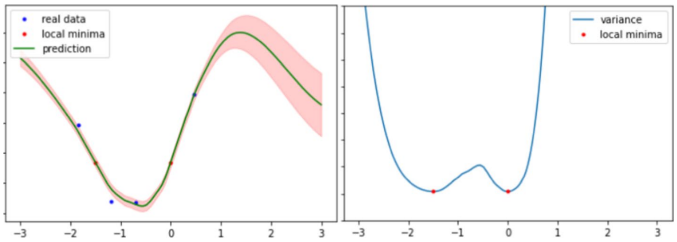

Fig. 3. Fig. 5 An example to show why variance adversarial attack fails when we have a non-stationary kernel. Left: training data and the predictive posterior of FedBNR. Right: the amount of variance for the predic tion in the left graph 1 3 72 Page 24 of 36 Machine Learning (2026) 115:72 unbounded, so we need to clip it by its max and min, and denote b := max(y) -min(y). Then we can flatten s(Xc, yc) = [ΦcΦ⊤ c , Φcyc, y⊤ c yc] and calculate its sensitivity:

√ m(2m + 1)(4/m)2 + 2m(2b/√m)2 + b4

(A1) L2(s) = = √ 32 + 16/m + 8b2 + b4

(A2) Note ΦcΦ⊤c is symmetric and we only need to protect the upper half triangular matrix, and this architecture results in 2m random features with m samples. Appendix B: Experiment Details Figure 6 shows the architecture of DNNs used in the real world regression experiments. For FedAvg, FedProx, and pFedGP, we used the left DNN as their feature extractor. For our methods, we used the rightmost architecture in Fig. 2 with the same feature extractor f, a very small distribution shifter h (the right DNN in Fig. 6), and ω ∼N(0, I5). We run each random seed of each dataset on 1 CPU and 1 NVIDIA T4 GPU with 16GB RAM. Some important hyperparameters are listed in Table 5. These hyperparameters are selected through grid searching, as suggested by FedProx. We make our code public in the supplementary materials, where other minutiae like the learning rate can be found. All datasets included along with our code are licensed under a Creative Commons Attribution 4.0 International (CC BY 4.0) license, which allows for the sharing and adaptation of the datasets for any purpose, provided that the appropriate credit is given. Appendix C: Additional Results Necessity of Random Features We perform an extra ablation study experiment similarly to the real-world experiment in generalization setting. Experiment details are the same as in Appendix B. Each setting is run for 10 different random seeds. RMSE reported in Table 6 shows that random features significantly improve the model capacity. Generalization Setting We assess models with testing data uniformly sampled globally in this setting. We include the maximum calibration error (MCE) and the Brier score (BRI) of the real world regression experiments in Tables 7 and 8.

MCE measures the (estimated) worst differ ence between p% confidence intervals and p'% test points falling into these intervals. BRI measures the mean squared difference between the confidence and observations, where a test point falling in the CI counts as 1, otherwise 0. We also include the standard error of the mean (SEM) in Table 9. Our methods still perform better in most of the cases. 1 3 Page 25 of 36 72 Machine Learning (2026) 115:72 Differential Privacy We report the trade-off between model utility and the privacy budget ϵ in Fig. 7. The y-axis reports RMSE, and the light area reports the standard error of the mean (SEM). Experiments were repeated for 5 different seeds. Other experiment details are similar to the generaliza tion setting. Personalization Setting We assess models with testing data uniformly sampled locally in this setting. We report MCE in Table 10 and BRI in Table 11. Appendix D: Unifying Random Kernel Approximate Deep Stationary Kernels We now prove Thm. 3.1 in the main paper. We make use of the Hoeffding's inequality (Hoeffding, 1994). Theorem A.1 (Hoeffding's inequality). Suppose X1, · · · , Xn are n independent random variables. Let ¯X = ∑n i=1 Xi/n. If ∀i ∈{1, · · · , n}, Pr(Xi ∈[ai, bi]) = 1 almost surely, then ∀t > 0 ( ) 2t2n2 ∑n Pr[| ¯X -E[¯X]| ≥t] ≤2 exp i=1(bi -ai)2 Theorem 3.1 ∀x, x' ∈Rp, ∀p(ω) being a proper distribution over Rd', and g : Rd' × Rp -→Rd has the form of Eq. 15. With m ∈Z+ samples of ω, ∀t > 0 ( ) -mt2 Pr[|URKm ω,g(x, x') -k(x, x')| ≥t] ≤2 exp 8 Proof Notice URKm ω,g(x, x') is the empirical estimation of k(x, x') = Eω[g(ω, x)⊤g(ω, x')]: Table 5 Hyperparameters used in the real world regression experiments. For FedLoc L = 10ρ to run the Proximal ADMM algorithm #Clients Skillcraft SML Parkinsons Bike CCPP 10 100 10 100 10 100 10 100 10 100 Size s 200 2000 2000 5000 5000 FedProx μ 0.5 1.0 0.1 1.0 1.0 1.0 0.1 0.01 0.001 1.0 FedLoc ρ 5e3 5e4 5e3 5e4 1e3 5e3 5e4 5e4 1e5 1e5 FedBNR-KD α 10 2 1 0.5 5 2 5 0.5 5 5

1 3 72 Page 26 of 36 Machine Learning (2026) 115:72 ( )⊤( )

VI. ∑

sm ω (x) √m sm ω (x') √m

VII. M(G(ωI, X)⊤G(ωI, X'))

URKm ω,g(x, x') = = i=1 g(ωi, x)⊤g(ωi, x') are i.i.d. 1-dimensional random variables. It is bounded by [-2, 2] since g(ω, x) = [sin(ω⊤ϕNN(x)), cos(ω⊤ϕNN(x))]⊤. Finally, apply the Hoeffding's inequality. □

Approximate Non-stationary Kernels Without DNN

Additionally, URK can be used to construct feature mappings of non-stationary kernels without the help of Deep Neural Networks. We prove the two examples we gave in the main paper now. Proposition 3.2 Let ω ∼N(0, I), g(ω, x) = exp(ω⊤x), then k(x, x') = limm-→∞URK ω,g(x, x') has infinite features. Proof k(x, x') = Eω[zω(x)⊤zω(x')] = Eω[exp (ω⊤(x + x'))] = Mω(x + x'), the moment generating function of ω = exp((x + x')⊤(x + x')/2) = exp((x⊤x + x'⊤x')/2) exp(x⊤x') ∞ ∑ (x⊤x')l = exp((x⊤x + x'⊤x')/2) l!

l=0 (exp(x⊤x/2) √) ∞ ∑ l! (x⊤x')l exp(x'⊤x'/2) √ = l!

l=0 = ϕ(x)⊤ϕ(x') □ Theorem 3.3 Let ω and g have the form of Prop. 3.2. Suppose x, x' ∈Rp satisfies ||x||2 2 ≤a, ||x'||2 2 ≤a'. With m ∈Z+ samples of ω, ∀t ∈(0, exp(a + a')/2)), by FentonWilkinson approximation, (exp (a + a') -1)

VIII. + 1

σs := log , E := exp((a + a')/2) Pr[|URKm ω,g(x, x') -k(x, x')| ≥t] ≤1 -F (2 log(t + E) -(a + a') + σ2 s 2σs) (2 log(E -t) -(a + a') + σ2 s 2σs) + F F is the cumulative probability distribution function of the standard normal distribution.

1 3 Page 27 of 36 72 Machine Learning (2026) 115:72 Proof Since ωi ∼N(0, I), ∀i and g(ωi, x)⊤g(ωi, x') = exp(ω⊤ i (x + x')), i (x + x') ∼N(0, σ2), σ2 ≤a + a' ω⊤ g(ωi, x)⊤g(ωi, x') ∼LogN(0, σ2), the log-normal distribution Since the sum of m i.i.d. log-normal distributions has no closed form, we use the FentonWilkinson approximation (Fenton, 1960) to model URKm ω,g(x, x'). It approximates the sum with a log-normal distribution by matching the first and second moments.

(exp(σ2) -1) σ2 s = log , E := k(x, x') = E[g(ω, x)⊤g(ω, x')] = exp(σ2/2)

IX. ∑

1 m(g(ωi, x)⊤g(ωi, x')) ∼LogN((σ2 -σ2 s)/2, σ2 s) URKm ω,g(x, x') = i=1 Pr[URKm ω,g(x, x') -E ≥t] = 1 -Pr[URKm ω,g(x, x') ≤t + E] = 1 -F (2 log(t + E) -σ2 + σ2 s 2σs) Pr[URKm ω,g(x, x') -E ≤-t] = Pr[URKm ω,g(x, x') ≤E -t] = F (2 log(E -t) -σ2 + σ2 s 2σs)

Pr[|URKm

ω,g(x, x') -k(x, x')| ≥t] = Pr[URKm ω,g(x, x') -E ≥t] + Pr[URKm ω,g(x, x') -E ≤-t] This equation takes its maximum when σ2 = a + a'. □ Proposition 3.4 Let ~c, ~n ∈R. Define ~p = [1 2 , 1 2p, 1 2p, · · · , 1 2p]⊤∈Rp+1, ω ∼Multi(~n, ~p), the multino mial distribution. Define ~x = [√ 2~c, √2px⊤]⊤∈Rp+1, g(ω, x) = exp(ω⊤log~x), then k(x, x') = limm-→∞URK ω,g(x, x') = (x⊤x' + ~c)~n Proof In the following proof, any subscript i means the ith entry of the vector. By definition, k(x, x') = Eω[zω(x)⊤zω(x')] = Eω[exp (ω⊤(log ~x + log ~x'))]

= Mω(log ~x + log ~x'), the moment generating function of ω (p+1 ∑)~n i=1 ppoly,i exp(log ~xi + log ~x' i) = (1 2 exp(2 log √)~n p ∑ 1 2p exp(log √ 2pxi + log √ = 2~c) + 2px' i) i=1 ( )~n p ∑ = ~c + i=1 xix' i = (x⊤x' + ~c)~n 1 3 72 Page 28 of 36 Machine Learning (2026) 115:72

□ Theorem 3.5 Let ω and g have the form of Prop. 3.4. Suppose entries of x, x' ∈Rp + are bounded by [a, b]. With m ∈Z+ samples of ω, ∀t > 0, √ 2~c, √2pb} min{ √ v := max{ 2~c, √2pa} (-2mt2) Pr[|URKm ω,g(x, x') -k(x, x')| ≥t] ≤2 exp v4~n Proof Similarly to Thm. 2, g(ωi, x)⊤g(ωi, x') are i.i.d. 1-dimensional random variables, bounded by [exp(~n log(min{ √ 2~c, √2pa})), exp(~n log(max{ √ 2~c, √2pb}))]. Then, apply Hoeffding's inequality. □ __IMAGE__intermediate/figures/fig_6.jpeg__PAGE_27__Y_247__ Fig. 6 Architecture of DNNs used in real world regression experiments. Left: the feature extractor for x; right: the distribution shifter for ω. FC is short for "fully connected". The layer size s varies for different datasets Table 6 RMSE

reported. Generalization setting. FedBNR\URK: removing random features from FedBNR; FedBNR: ordinary FedBNR. ⇑ and ↑ denote significantly worse results with p < 0.01 and p < 0.05 respec tively; ⇓ and ↓ denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP 10 100 10 100 10 100 10 100 10 100

FedBNR\URK 1.15⇑ 1.20⇑ 0.26↑ 0.64⇑ 5.95⇑ 6.72⇑ 0.40 0.43⇑ 4.23⇓ 4.16⇓

FedBNR 1.00 0.97 0.24 0.52 3.16 5.33 0.40 0.41 4.41 4.51 Table 7 Maximum calibration error (MCE) reported. Generalization setting. ⇑ and ↑ denote significantly worse results with p < 0.01 and p < 0.05 respectively; ⇓ and ↓ denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP 10 100 10 100 10 100 10 100 10 100 Central GP 0.03 0.16 0.42 0.16 0.40 pFedGP 0.77⇑ 0.65⇑ 0.82⇑ 0.83⇑ 0.93⇑ 0.91⇑ 0.78⇑ 0.73⇑ 0.94⇑ 0.93⇑

FedLoc 0.51⇑ 0.81⇑ 0.20⇓ 0.44⇑ 0.54⇑ 0.76⇑ 0.42⇑ 0.28⇑ 0.39⇓ 0.95⇑

DEC-apx 0.95⇑ 0.80⇑ 0.95⇑ 0.95⇑ 0.95⇑ 0.95⇑ 0.34⇑ 0.29⇑ 0.74⇑ 0.65⇑ DEC-gapx 0.95⇑ 0.73⇑ 0.95⇑ 0.95⇑ 0.95⇑ 0.95⇑ 0.44⇑ 0.32⇑ 0.84⇑ 0.75⇑ ours FedBNR 0.09 0.32⇑ 0.67⇑ 0.62⇑ 0.62⇑ 0.72⇑ 0.15 0.64⇑ 0.39⇓ 0.31⇓ FedBNR-KD 0.10 0.11 0.33 0.35 0.47 0.48 0.15 0.17 0.49 0.50 Table 8 Brier score (BRI) reported. Generalization setting. ⇑ and ↑ denote significantly worse results with p < 0.01 and p < 0.05 respectively; ⇓ and ↓ denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP 10 100 10 100 10 100 10 100 10 100 Central GP 0.16 0.20 0.24 0.18 0.24 pFedGP 0.30⇑ 0.28⇑ 0.31⇑ 0.31⇑ 0.33⇑ 0.33⇑ 0.30⇑ 0.30⇑ 0.33⇑ 0.33⇑

FedLoc 0.21 0.31⇑ 0.18⇓ 0.25⇑ 0.26⇑ 0.30⇑ 0.19↓ 0.16⇓ 0.24⇓ 0.33⇑

DEC-apx 0.33⇑ 0.31⇑ 0.33⇑ 0.33⇑ 0.33⇑ 0.33⇑ 0.17⇓ 0.15⇓ 0.27⇑ 0.24⇓ DEC-gapx 0.33⇑ 0.28⇑ 0.33⇑ 0.33⇑ 0.33⇑ 0.33⇑ 0.20⇑ 0.26⇑ 0.30⇑ 0.26 ours FedBNR 0.18 0.22⇑ 0.29⇑ 0.28⇑ 0.28⇑ 0.30⇑ 0.20⇑ 0.29⇑ 0.24⇓ 0.22⇓ FedBNR-KD 0.18 0.18 0.23 0.23 0.25 0.26 0.19 0.20 0.25 0.26 DEC-apx 1.10 ± 0.01 4.34 ± 0.01 1.90 ± 0.04 4.93 ± 0.08 8.59 ± 0.07 9.21 ± 0.03 0.76 ± 0.01 0.88 ± 0.01 7.64 ± 0.28 11.06 ± 0.14 FedBNR-KD 0.96 ± 0.01 0.98 ± 0.01 0.55 ± 0.01 0.55 ± 0.01 4.58 ± 0.05 4.67 ± 0.04 0.43 ± 0.01 0.48 ± 0.01 4.38 ± 0.01 4.38 ± 0.01 Local+local 1.26 ± 0.03 1.48 ± 0.01 1.49 ± 0.06 2.32 ± 0.03 10.9 ± 0.26 10.6 ± 0.05 0.79 ± 0.02 0.91 ± 0.01 14.6 ± 0.31 19.3 ± 2.08 Local+global 1.08 ± 0.02 1.22 ± 0.01 1.00 ± 0.02 1.65 ± 0.01 6.42 ± 0.10 7.42 ± 0.01 0.59 ± 0.01 0.73 ± 0.01 5.62 ± 0.19 7.03 ± 0.16 Avg+local 1.05 ± 0.02 1.06 ± 0.02 0.61 ± 0.06 0.81 ± 0.05 9.84 ± 0.31 8.80 ± 0.11 0.45 ± 0.01 0.54 ± 0.01 8.32 ± 0.43 13.4 ± 0.43 kd+local 1.04 ± 0.01 1.28 ± 0.02 0.86 ± 0.06 1.34 ± 0.12 6.15 ± 0.30 6.97 ± 0.27 0.51 ± 0.01 0.61 ± 0.01 10.0 ± 0.61 17.1 ± 0.63 FedAvg 1.00 ± 0.01 1.03 ± 0.01 0.36 ± 0.02 0.71 ± 0.02 6.83 ± 0.16 7.38 ± 0.21 0.38 ± 0.01 0.42 ± 0.01 4.45 ± 0.06 4.44 ± 0.04 FedProx 0.98 ± 0.01 1.05 ± 0.01 0.34 ± 0.01 0.62 ± 0.02 6.32 ± 0.27 7.38 ± 0.25 0.39 ± 0.01 0.42 ± 0.01 4.43 ± 0.04 4.47 ± 0.02 pFedGP 0.99 ± 0.01 1.15 ± 0.01 0.75 ± 0.05 1.34 ± 0.04 9.36 ± 0.16 8.91 ± 0.09 0.45 ± 0.02 0.46 ± 0.01 14.2 ± 0.61 18.2 ± 0.23 FedLoc 1.08 ± 0.01 4.15 ± 0.26 2.83 ± 0.07 5.43 ± 0.04 8.40 ± 0.02 11.5 ± 0.03 0.64 ± 0.01 0.75 ± 0.01 20.6 ± 0.74 44.1 ± 0.61 DEC-gapx 0.94 ± 0.01 4.31 ± 0.01 1.41 ± 0.01 3.57 ± 0.09 8.07 ± 0.02 8.26 ± 0.01 0.55 ± 0.01 0.75 ± 0.01 6.26 ± 0.04 9.12 ± 0.22 FedBNR 0.98 ± 0.01 0.97 ± 0.01 0.25 ± 0.01 0.44 ± 0.02 3.10 ± 0.23 5.42 ± 0.20 0.39 ± 0.01 0.42 ± 0.01 4.40 ± 0.01 4.51 ± 0.03 10 100 10 100 10 100 10 100 10 100 #Clients Skillcraft SML Parkinsons Bike CCPP Table 9 RMSE ± standard error of the mean (SEM) reported. Generalization setting

ours __IMAGE__intermediate/figures/fig_7.png__PAGE_30__Y_53__ Fig. 7 FedBNR+DP results. Phase 1 DP: add DP protection to model parameters θ only; full DP: add DP protection to all messages Table 10 Maximum calibration error (MCE) reported. Personalization setting. ⇑ and ↑ denote significantly worse results with p < 0.01 and p < 0.05 respectively; ⇓ and ↓ denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP 10 100 10 100 10 100 10 100 10 100 pFedGP 0.76⇑ 0.59⇑ 0.82⇑ 0.80⇑ 0.83⇑ 0.81⇑ 0.77⇑ 0.68⇑ 0.93⇑ 0.93⇑

pxADMM 0.76⇑ 0.58⇑ 0.94⇑ 0.83⇑ 0.93⇑ 0.81⇑ 0.30⇑ 0.34⇑ 0.94⇑ 0.93⇑

DEC-apx 0.95⇑ 0.95⇑ 0.94⇑ 0.95⇑ 0.92⇑ 0.95⇑ 0.44⇑ 0.39⇑ 0.43⇑ 0.76⇑ DEC-gapx 0.95⇑ 0.95⇑ 0.95⇑ 0.95⇑ 0.80⇑ 0.92⇑ 0.46⇑ 0.42⇑ 0.35⇓ 0.23⇓ ours FedBNR 0.03 0.06 0.40 0.48 0.08 0.10 0.19 0.14 0.41 0.56 FedBNR-KD 0.03 0.05↓ 0.25⇓ 0.30⇓ 0.12⇑ 0.17⇑ 0.19 0.12 0.47⇑ 0.55 Table 11 Brier score (BRI) reported. Personalization setting. ⇑ and ↑ denote significantly worse results with p < 0.01 and p < 0.05 respectively; ⇓ and ↓ denote significantly better results similarly #Clients Skillcraft SML Parkinsons Bike CCPP 10 100 10 100 10 100 10 100 10 100 pFedGP 0.30⇑ 0.27⇑ 0.31⇑ 0.31⇑ 0.31⇑ 0.31⇑ 0.30⇑ 0.29⇑ 0.33⇑ 0.33⇑

pxADMM 0.30⇑ 0.27⇑ 0.33⇑ 0.31⇑ 0.33⇑ 0.31⇑ 0.22⇑ 0.23⇑ 0.33⇑ 0.33⇑ DEC-apx 0.33⇑ 0.33⇑ 0.33⇑ 0.33⇑ 0.33⇑ 0.33⇑ 0.20⇑ 0.17⇓ 0.24 0.27 DEC-gapx 0.33⇑ 0.33⇑ 0.33⇑ 0.33⇑ 0.30⇑ 0.33⇑ 0.20⇑ 0.18⇓ 0.16⇓ 0.16⇓ ours FedBNR 0.17 0.18 0.24 0.25 0.18 0.19 0.19 0.20 0.24 0.27 FedBNR-KD 0.17 0.18↓ 0.21⇓ 0.22⇓ 0.19⇑ 0.20⇑ 0.19 0.19⇓ 0.25⇑ 0.27 Supplementary Information The online version contains supplementary material available at https://doi.org /10.1007/s10994-025-06936-5. Acknowledgements Resources used in preparing this research at the University of Waterloo were provided by Huawei Canada, the province of Ontario and the government of Canada through CIFAR and companies sponsoring the Vector Institute.

Author Contributions H.Y., K.G., M.K., G.Z., and P.P. contributed to the study conception and design. Mate rial preparation, data collection and analysis were performed by H.Y. The first draft of the manuscript was written by H.Y. and P.P. All

authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding The research leading to these results received funding from Huawei Canada under Grant Agreement #TC20210831001 and the AI Chair program of the Canadian Institute for Advanced Research (CIFAR).

Data Availability No datasets were generated or analysed during the current study.

Materials Availability Not applicable.

Code Availability We provide public code in the supplementary file.

Declarations

Conflict of interest The authors declare no Conflict of interest.

Ethical Approval Not applicable.

Consent to Participate Not applicable.

Consent for Publication All authors consent to the submission of this manuscript to the Machine Learning.

References

[1] Achituve, I., Shamsian, A., Navon, A., Chechik, G., & Fetaya, E. (2021). Personalized federated learning with Gaussian processes. Advances in Neural Information Processing Systems, 34, 8392–8406. Begoli, E., Bhattacharya, T., & Kusnezov, D. (2019). The need for uncertainty quantification in machine- assisted medical decision making. Nature Machine Intelligence, 1(1), 20–23. Boenisch, F., Dziedzic, A., Schuster, R., Shamsabadi, A. S., Shumailov, I., & Papernot, N. (2021). When the curious abandon honesty: Federated learning is not private. arXiv preprint arXiv:2112.02918. Byrd, D., & Polychroniadou, A. (2020). Differentially private secure multi-party computation for federated learning in financial applications. In: Proceedings of the first ACM international conference on AI in Finance, pp. 1–9. Calandra, R., Peters, J., Rasmussen, C. E., & Deisenroth, M. P. (2016). Manifold Gaussian processes for regression. In: 2016 International joint conference on neural networks (IJCNN), pp. 3338–3345. IEEE. Cao, Y., & Fleet, D. J. (2014). Generalized product of experts for automatic and principled fusion of Gauss ian process predictions. arXiv preprint arXiv:1410.7827. Cuturi, M. (2009). Positive definite kernels in machine learning. arXiv preprint arXiv:0911.5367. Damianou, A., & Lawrence, N.D. (2013). Deep gaussian processes. In: Artificial intelligence and statistics, pp. 207–215. PMLR. Danquah, B., Riedmaier, S., Rühm, J., Kalt, S., & Lienkamp, M. (2020). Statistical model verification and validation concept in automotive vehicle design. Procedia Cirp, 91, 261–270. Deisenroth, M., & Ng, J.W. (2015). Distributed gaussian processes. In: International conference on machine learning, pp. 1481–1490. PMLR. 1 3 Page 33 of 36 72 Machine Learning (2026) 115:72 Fenton, L. (1960). The sum of log-normal probability distributions in scatter transmission systems. IRE Transactions on Communications Systems, 8(1), 57–67. Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In: International Conference on Machine Learning, pp. 1050–1059. PMLR. Gal, Y., Van Der Wilk, M., & Rasmussen, C. E. (2014). Distributed variational inference in sparse Gaussian process regression and latent variable models. Advances in Neural Information Processing Systems, 27. Garnelo, M., Rosenbaum, D., Maddison, C., Ramalho, T., Saxton, D., Shanahan, M., Teh, Y. W., Rezende, D., & Eslami, S.A. (2018). Conditional neural processes. In: International conference on machine learning, pp. 1704–1713. PMLR. Gawlikowski, J., Tassi, C. R. N., Ali, M., Lee, J., Humt, M., Feng, J., Kruspe, A., Triebel, R., Jung, P., Roscher, R., Shahzad, M., Yang, W., Bamler, R., & Zhu, X. X. (2022). A survey of uncertainty in deep neural networks. He, B., & Ozay, M. (2022). Feature kernel distillation. In: International conference on learning representations. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770–778. Hensman, J., Fusi, N., & Lawrence, N. D. (2013). Gaussian processes for big data. arXiv preprint arXiv:1309.6835. Hinton, G. E., & Salakhutdinov, R. R. (2007). Using deep belief nets to learn covariance kernels for Gaussian processes. Advances in Neural Information Processing Systems, 20. Hoang, T. N., Hoang, Q. M., & Low, B. K. H. (2016). A distributed variational inference framework for unifying parallel sparse Gaussian process regression models. In: International conference on machine learning, pp. 382–391. PMLR. Hoeffding, W. (1994). Probability inequalities for sums of bounded random variables. The collected works of Wassily Hoeffding, 409–426. Iyer, A., Zhang, Y., Prasad, A., Gupta, P., Tao, S., Wang, Y., Prabhune, P., Schadler, L. S., Brinson, L. C., & Chen, W. (2020). Data centric nanocomposites design via mixed-variable Bayesian optimization. Molecular Systems Design & Engineering, 5(8), 1376–1390. Iyer, A., Zhang, Y., Prasad, A., Tao, S., Wang, Y., Schadler, L., Brinson, L. C., & Chen, W. (2019). Data- centric mixed-variable Bayesian optimization for materials design. In: International design engineer ing technical conferences and computers and information in engineering conference, vol. 59186, pp. 02–03066. American Society of Mechanical Engineers. Karimireddy, S. P., Kale, S., Mohri, M., Reddi, S., Stich, S., & Suresh, A. T. (2020). Scaffold: Stochas tic controlled averaging for federated learning. In: International conference on machine learning, pp. 5132–5143. PMLR Kerkouche, R., Ács, G., Castelluccia, C., & Genevès, P. (2021). Privacy-preserving and bandwidth-efficient federated learning: An application to in-hospital mortality prediction. In: Proceedings of the conference on health, inference, and learning. CHIL '21, pp. 25–35. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3450439.3451859. Konečnỳ, J., McMahan, H. B., Yu, F. X., Richtárik, P., Suresh, A. T., & Bacon, D. (2016). Federated learn ing: Strategies for improving communication efficiency. arXiv preprint arXiv:1610.05492. Kontoudis, G. P., & Stilwell, D. J. (2023). Decentralized federated learning using gaussian processes. In: 2023 International symposium on multi-robot and multi-agent systems (MRS), pp. 1–7. IEEE. Kontoudis, G. P., & Stilwell, D. J. (2024). Scalable, federated gaussian process training for decentralized multi-agent systems. IEEE Access, 12, 77800–77815. Kristiadi, A., Hein, M., & Hennig, P. (2020). Being Bayesian, even just a bit, fixes overconfidence in ReLU networks. Lederer, A., Umlauft, J., & Hirche, S. (2019). Uniform error bounds for gaussian process regression with application to safe control. Advances in Neural Information Processing Systems, 32. Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated optimization in heterogeneous networks. Proceedings of Machine Learning and Systems, 2, 429–450. Li, Y., Rao, S., Hassaine, A., Ramakrishnan, R., Canoy, D., Salimi-Khorshidi, G., Mamouei, M., Lukasie wicz, T., & Rahimi, K. (2021). Deep Bayesian gaussian processes for uncertainty estimation in elec tronic health records. Scientific Reports, 11(1), 20685. Li, Y., Zhou, Y., Jolfaei, A., Yu, D., Xu, G., & Zheng, X. (2020). Privacy-preserving federated learning frame work based on chained secure multiparty computing. IEEE Internet of Things Journal, 8(8), 6178–6186. Liu, H., Ong, Y.-S., Shen, X., & Cai, J. (2020). When Gaussian process meets big data: A review of scalable gps. IEEE Transactions on Neural Networks and Learning Systems, 31(11), 4405–4423. McMahan, B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. (2017). Communication-efficient learn ing of deep networks from decentralized data. In: Artificial intelligence and statistics, pp. 1273–1282. PMLR. 1 3 72
Page 34 of 36 Machine Learning (2026) 115:72 Mohri, M., Sivek, G., & Suresh, A. T. (2019). Agnostic federated learning. In: International conference on machine learning, pp. 4615–4625. PMLR. Mothukuri, V., Parizi, R. M., Pouriyeh, S., Huang, Y., Dehghantanha, A., & Srivastava, G. (2021). A survey on security and privacy of federated learning. Future Generation Computer Systems, 115, 619–640. Nickson, T., Gunter, T., Lloyd, C., Osborne, M. A., & Roberts, S. (2015). Blitzkriging: Kronecker-structured stochastic Gaussian processes. arXiv preprint arXiv:1510.07965. Ober, S. W., Rasmussen, C. E., & Wilk, M. (2021). The promises and pitfalls of deep kernel learning. In: Uncertainty in artificial intelligence, pp. 1206–1216. PMLR. Oliva, J. B., Dubey, A., Wilson, A. G., Póczos, B., Schneider, J., & Xing, E. P. (2016). Bayesian nonparamet ric kernel-learning. In: Artificial intelligence and statistics, pp. 1078–1086. PMLR. Quinonero-Candela, J., & Rasmussen, C. E. (2005). A unifying view of sparse approximate Gaussian process regression. The Journal of Machine Learning Research, 6, 1939–1959. Rahimi, A., & Recht, B. (2007). Random features for large-scale kernel machines. Advances in Neural Infor mation Processing Systems, 20. Reisizadeh, A., Mokhtari, A., Hassani, H., Jadbabaie, A., & Pedarsani, R. (2020). Fedpaq: A communication- efficient federated learning method with periodic averaging and quantization. In: International confer ence on artificial intelligence and statistics, pp. 2021–2031. PMLR. Ried-

maier, S., Danquah, B., Schick, B., & Diermeyer, F. (2021). Unified framework and survey for model verification, validation and uncertainty quantification. Archives of Computational Methods in Engineer ing, 28, 2655–2688. Riedmaier, S., Ponn, T., Ludwig, D., Schick, B., & Diermeyer, F. (2020). Survey on scenario-based safety assessment of automated vehicles. IEEE Access, 8, 87456–87477. Rudin, W. (2017). Fourier analysis on groups. Garden City: Courier Dover Publications. Saatçi, Y. (2012). Scalable inference for structured Gaussian process models. PhD thesis, Citeseer. Sattler, F., Wiedemann, S., Müller, K.-R., & Samek, W. (2019). Robust and communication-efficient feder ated learning from non-iid data. IEEE Transactions on Neural Networks and Learning Systems, 31(9), 3400–3413. Schwaighofer, A., & Tresp, V. (2002). Transductive and inductive methods for approximate Gaussian process regression. Advances in Neural Information Processing Systems, 15. Seeger, M. W., Williams, C. K., & Lawrence, N. D. (2003). Fast forward selection to speed up sparse Gauss ian process regression. In: International workshop on artificial intelligence and statistics, pp. 254–261. PMLR. Shoham, N., Avidor, T., Keren, A., Israel, N., Benditkis, D., Mor-Yosef, L., & Zeitak, I. (2019). Overcoming forgetting in federated learning on non-iid data. arXiv preprint arXiv:1910.07796. Sinha, A., & Duchi, J. C. (2016). Learning kernels with random features. Advances in Neural Information Processing Systems, 29. Smith, M. T., Álvarez, M. A., & Lawrence, N. D. (2021). Differentially private regression and classification with sparse Gaussian processes. Journal of Machine Learning Research, 22, 188. Smola, A., & Bartlett, P. (2000). Sparse greedy Gaussian process regression. Advances in Neural Information Processing Systems, 13. Snelson, E., & Ghahramani, Z. (2005). Sparse Gaussian processes using pseudo-inputs. Advances in Neural Information Processing Systems, 18. Tang, M., Ning, X., Wang, Y., Wang, Y., & Chen, Y. (2021). Fedgp: Correlation-based active client selection strategy for heterogeneous federated learning. arXiv preprint arXiv:2103.13822. Thompson, J. J., Blair, M. R., Chen, L., & Henrey, A. J. (2013). Video game telemetry as a critical tool in the study of complex skill learning. PLoS ONE, 8(9), 75129. Titsias, M. (2009). Variational learning of inducing variables in sparse Gaussian processes. In: Artificial intel ligence and statistics, pp. 567–574. PMLR. Tran, G.-L., Bonilla, E. V., Cunningham, J., Michiardi, P., & Filippone, M. (2019). Calibrating deep convolu tional Gaussian processes. In: The 22nd international conference on artificial intelligence and statistics, pp. 1554–1563. PMLR. Triastcyn, A., & Faltings, B. (2019). Federated learning with bayesian differential privacy. In: 2019 IEEE international conference on big data (Big Data), pp. 2587–2596. IEEE. Truex, S., Baracaldo, N., Anwar, A., Steinke, T., Ludwig, H., Zhang, R., & Zhou, Y. (2019). A hybrid approach to privacy-preserving federated learning. In: Proceedings of the 12th ACM workshop on artificial intel ligence and security, pp. 1–11. Tsanas, A., Little, M., McSharry, P., & Ramig, L. (2009). Accurate telemonitoring of Parkinson's disease progression by non-invasive speech tests. Nature Proceedings pp. 1–1. Tüfekci, P. (2014). Prediction of full load electrical power output of a base load operated combined cycle power plant using machine learning methods. International Journal of Electrical Power & Energy Systems, 60, 126–140. 1 3 Page 35 of 36 72 Machine Learning (2026) 115:72 Ve, S., & Cho, Y. (2020). A rule-based model for Seoul bike sharing demand prediction using weather data. European Journal of Remote Sensing, 53(sup1), 166–183. Verrelst, J., Rivera, J. P., Moreno, J., & Camps-Valls, G. (2013). Gaussian processes uncertainty estimates in experimental sentinel-2 lai and leaf chlorophyll content retrieval. ISPRS Journal of Photogrammetry and Remote Sensing, 86, 157–167. Wei, K., Li, J., Ding, M., Ma, C., Yang, H. H., Farokhi, F., Jin, S., Quek, T. Q., & Poor, H. V. (2020). Feder ated learning with differential privacy: Algorithms and performance analysis. IEEE Transactions on Information Forensics and Security, 15, 3454–3469. Wilcoxon, F. (1992). Individual comparisons by ranking methods. In: Breakthroughs in statistics, pp. 196– 202. Springer. Wilson, A. G., Dann, C., & Nickisch, H. (2015). Thoughts on massively scalable Gaussian processes. arXiv preprint arXiv:1511.01870. Wilson, A. G., Hu, Z., Salakhutdinov, R. R., & Xing, E. P. (2016a). Stochastic variational deep kernel learn ing. Advances in Neural Information Processing Systems, 29, 2586–2594. Wilson, A. G., Hu, Z., Salakhutdinov, R., & Xing, E. P. (2016b). Deep kernel learning. In: Artificial intel ligence and statistics, pp. 370–378. PMLR. Wilson, A., & Nickisch, H. (2015). Kernel interpolation for scalable structured Gaussian processes (kiss-gp). In: International conference on machine learning, pp. 1775–1784. PMLR. Xia, X., Sivonxay, E., Helms, B. A., Blau, S. M., & Chan, E. M. (2023). Accelerating the design of multishell upconverting nanoparticles through Bayesian optimization. Nano Letters, 23(23), 11129–11136. Xie, A., Yin, F., Xu, Y., Ai, B., Chen, T., & Cui, S. (2019). Distributed Gaussian processes hyperparameter optimization for big data using proximal admm. IEEE Signal Processing Letters, 26(8), 1197–1201. Yin, F., Lin, Z., Kong, Q., Xu, Y., Li, D., Theodoridis, S., & Cui, S. R. (2020). Fedloc: Federated learning framework for data-driven cooperative localization and location data processing. IEEE Open Journal of Signal Processing, 1, 187–215. Yurochkin, M., Agarwal, M., Ghosh, S., Greenewald, K., & Hoang, N. (2019). Statistical model aggregation via parameter matching. Advances in Neural Information Processing Systems, 32. Zamora-Martinez, F., Romeu, P., Botella-Rocamora, P., & Pardo, J. (2014). On-line learning of indoor tem perature forecasting models towards energy efficiency. Energy and Buildings, 83, 162–172. Zhu, H., Xu, J., Liu, S., & Jin, Y. (2021). Federated learning on non-iid data: A survey. Neurocomputing, 465, 371–390. Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations. Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manu script version of this article is solely governed by the terms of such publishing agreement and applicable law. Authors and Affiliations Haolin Yu1,2 · Kaiyang Guo3 · Mahdi Karami1 · Guojun Zhang4 · Hongliang Li3 · Xi Chen3 · Pascal Poupart1,2 Pascal Poupart ppoupart@uwaterloo.ca Haolin Yu h89yu@uwaterloo.ca Kaiyang Guo guokaiyang@huawei.com Mahdi Karami mahdikaramia@gmail.com Guojun Zhang guojun.zhang@uwaterloo.ca 1 3 72
 Hongliang Li hongliang.li2@huawei.com Xi Chen xi.chen11@mcgill.ca 1 University of Waterloo, Waterloo, ON, Canada 2 Vector Institute for AI, Toronto, ON, Canada 3 Noah's Ark Lab, Huawei Technologies, Shenzhen, China 4 MiniMax, Beijing, China 1 3